

Python Assignment 2
William El Gharib
R232153H

```
① Import sqlalchemy
connection = sqlalchemy.create_engine('school.db')
cursor = connection.cursor()
cursor.execute('create table if not exists students (id integer primary
key autoincrement, name text not null, age integer not null, course text
not null)')
```

```
students = {
    Dave, 20, Business Management
    Billy, 22, Information Technology
}
cursor.executemany('insert into students (name, age, course)
connection.commit()')
```

```
cursor.execute('select id, name, age, course from students')
records = cursor.fetchall()
```

```
print('Student records:')
```

```
for record in records:
```

```
    print('')
```

```
    f ID: {record[0]}
```

```
    f Name: {record[1]}
```

```
    f Age: {record[2]}
```

```
    f Course: {record[3]}
```

```
connection.close()
```

```
print('Database connection closed')
```

2) class BankAccount

def __init__(self, balance=0)

self.__balance = balance

def deposit(self, amount)

if amount > 0

self.__balance += amount

print(f Deposited : {amount})

else

print (Deposited amount must be positive)

def withdraw(self, amount)

if 0 < amount <= self.__balance

self.__balance -= amount

print (f withdraw : {amount})

else

print (Insuficient withdrawl amount)

def get balance (self):

return self.__balance

def get_account_number (self):

return self.__account_number

account = BankAccount(123456789)

account.deposit(500)

account.withdraw(100)

print (f Current Balance : {account.get_balance()})

4. 2.)

```
numbers = [random.uniform(0, 10) for _ in range(5)]
```

```
print (Generated numbers : )
```

```
for number in numbers :
```

```
    print (f {number : 2f})
```

```
minimum_value = min(numbers)
```

```
maximum_value = max(numbers)
```

```
print (f In minimum value : {minimum_value : 2f})
```

```
print (f maximum value : {maximum_value : 2f})
```

6) class Vehicle :

```
    def __init__(self, brand)
```

```
        self.brand = brand
```

```
    def move(self) :
```

```
        print(f {self.brand} vehicle is moving)
```

```
class Car (Vehicle) :
```

```
    def move (self) :
```

```
        print(f {self.brand} car is driving on the road)
```

```
class Bike (Vehicle) :
```

```
    def move (self) :
```

```
        print(f {self.brand} bike is riding on the road)
```

```
vehicle = Vehicle (Generic)
```

```
car = Car (Toyota)
```

```
bike = Bike (Yamaha)
```

vehicle.move()

car.move()

bike.move()

5) from abc import ABC, abstractmethod

class FileHandler(ABC):

def read(self):

read data from a file

pass

def write(self, data):

write data to a file

pass

class TextFileHandler(FileHandler):

def __init__(self, filename):

self.filename = filename

def read(self):

try:

with open(self.filename, "r", encoding = utf-8) as file:
return file.read()

except FileNotFoundError:

return Text file was not found

def write(self, data):

with open(self.filename, "w", encoding = utf-8) as file:
file.write(data)

print(f"Text data written to {self.filename}").

```
class BinaryFileHandler(FileHandler):  
    def __init__(self, file_name):  
        self.filename = file_name
```

```
    def read(self):
```

```
        try:
```

```
            with open(self.filename, 'rb') as file:
```

```
                return file.read()
```

```
        except FileNotFoundError:
```

```
            return b'Binary file was not found'
```

```
    def write(self, data):
```

```
        if not isinstance(data, bytes):
```

```
            with open(self.filename, 'wb') as file:
```

```
                file.write(data)
```

```
        print(f'Binary data written to {self.filename}').
```